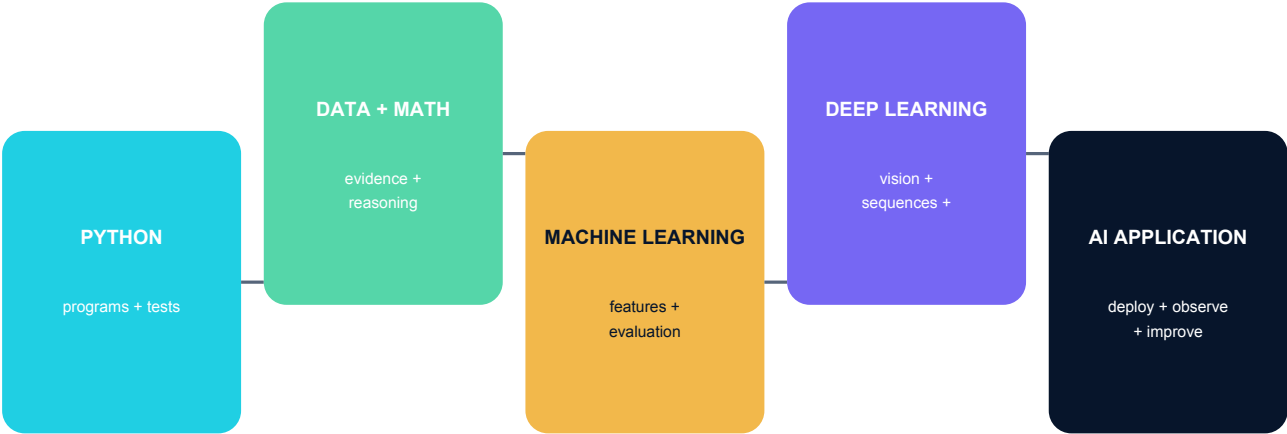


VISUAL FIELD GUIDE

Twelve reusable architecture and learning diagrams for lecture downloads, revision, presentations, and project documentation.

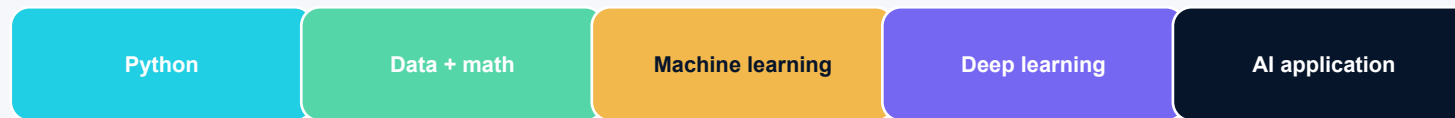


15 WEEKS	125 LECTURES	FULL STACK
12 visual systems	Professional resource edition	Learn + build + explain

USE DIAGRAMS TO REASON

A good technical diagram exposes sequence, boundaries, state, and feedback. Ask students to redraw each visual from memory, annotate the contracts, and add the metric or failure mode that matters at every transition.

FULL-STACK LEARNING JOURNEY

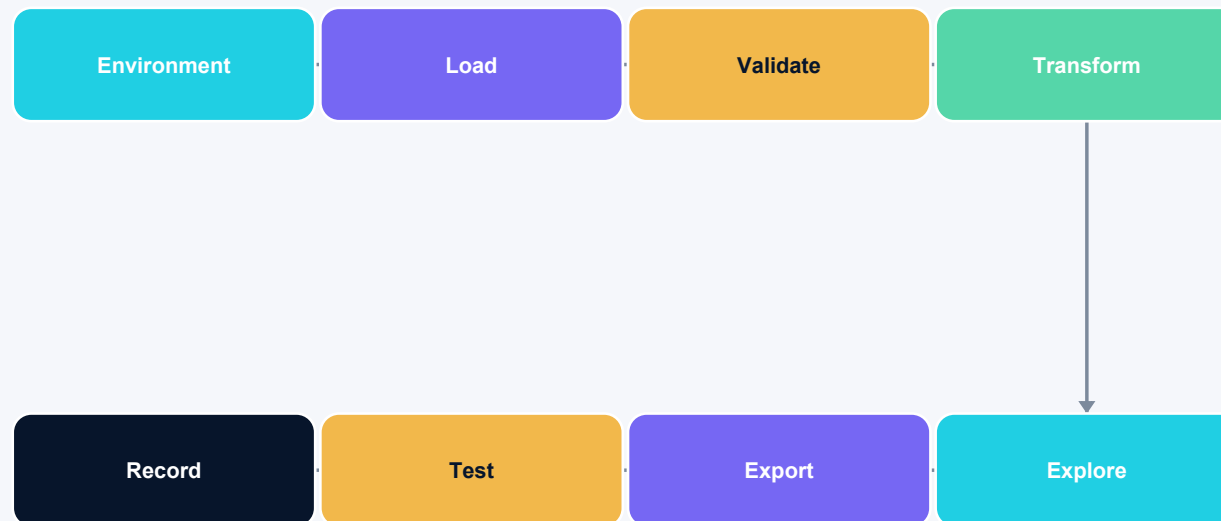


Each layer adds a new contract; a full-stack engineer can trace evidence across all of them.

TEACHING PROMPT

For each arrow: name what moves, its shape or schema, who owns it, how it can fail, and which measurement proves the handoff works.

REPRODUCIBLE DATA WORKFLOW

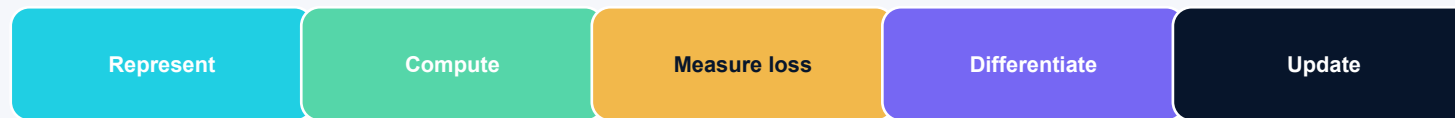


Keep schema, transformations, versions, tests, and findings in the same reproducible path.

TEACHING PROMPT

For each arrow: name what moves, its shape or schema, who owns it, how it can fail, and which measurement proves the handoff works.

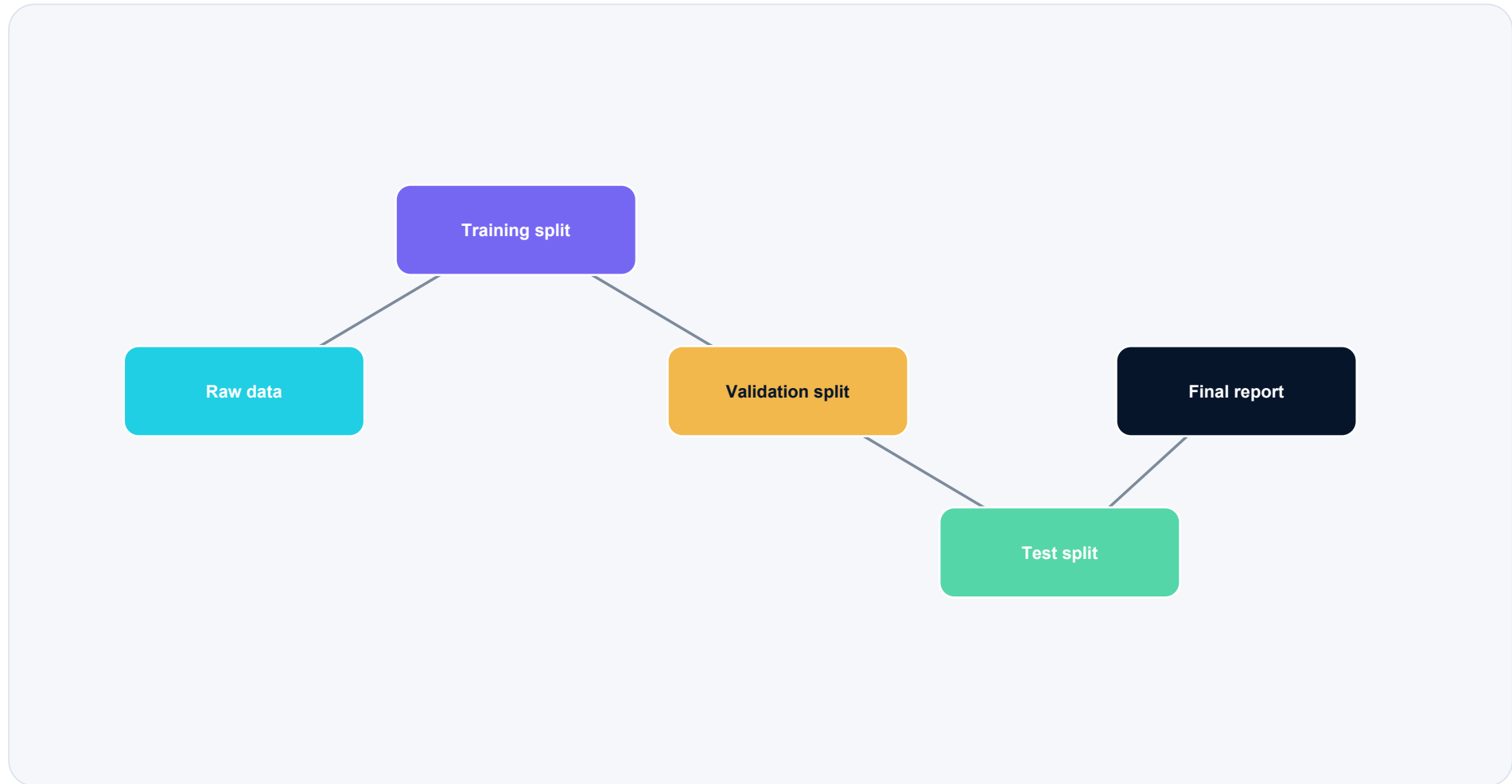
MATH TO MODEL REASONING



Linear algebra describes the computation; calculus describes how parameters should change.

TEACHING PROMPT

For each arrow: name what moves, its shape or schema, who owns it, how it can fail, and which measurement proves the handoff works.

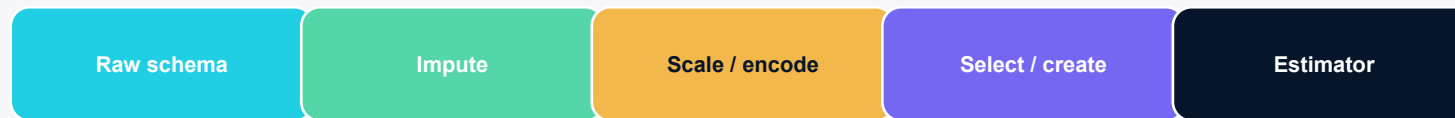


Fit on training data, choose with validation evidence, and touch the test set once for an honest estimate.

TEACHING PROMPT

For each arrow: name what moves, its shape or schema, who owns it, how it can fail, and which measurement proves the handoff works.

FEATURE ENGINEERING PIPELINE

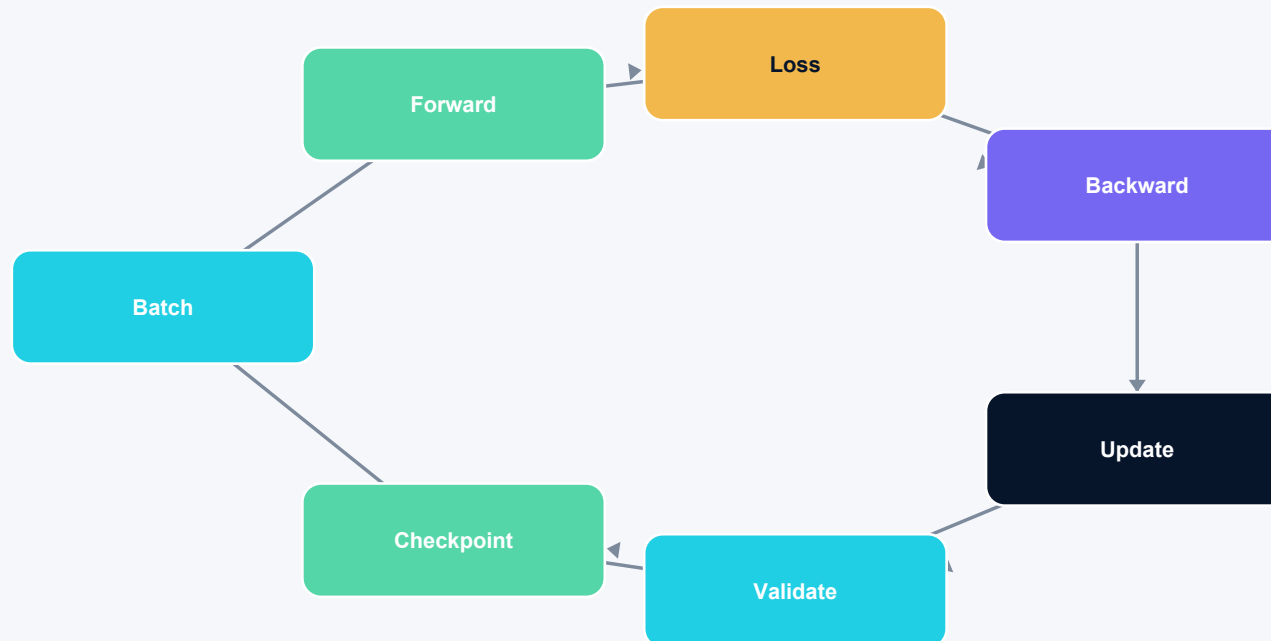


Fit every learned transformation inside the training pipeline to prevent leakage and train/serve skew.

TEACHING PROMPT

For each arrow: name what moves, its shape or schema, who owns it, how it can fail, and which measurement proves the handoff works.

NEURAL NETWORK LEARNING LOOP

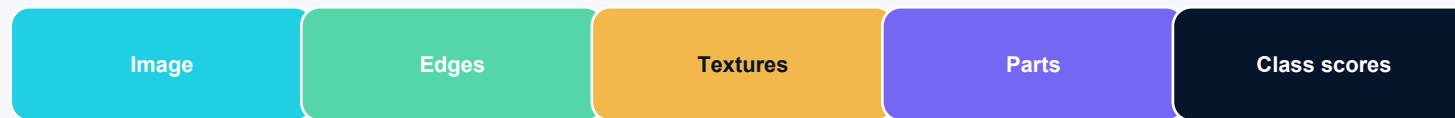


A stable training loop controls mode, gradients, optimizer state, evaluation, and checkpoint selection.

TEACHING PROMPT

For each arrow: name what moves, its shape or schema, who owns it, how it can fail, and which measurement proves the handoff works.

CNN FEATURE HIERARCHY

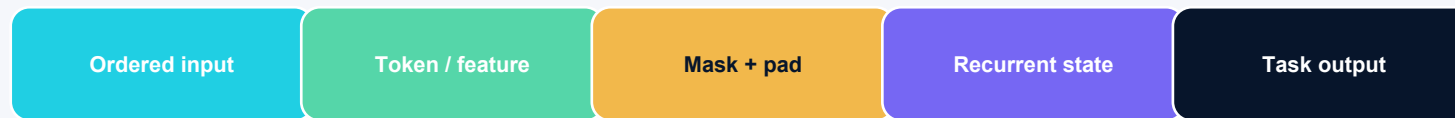


Convolution builds increasingly abstract spatial features while sharing weights across locations.

TEACHING PROMPT

For each arrow: name what moves, its shape or schema, who owns it, how it can fail, and which measurement proves the handoff works.

SEQUENCE MODELING PATH

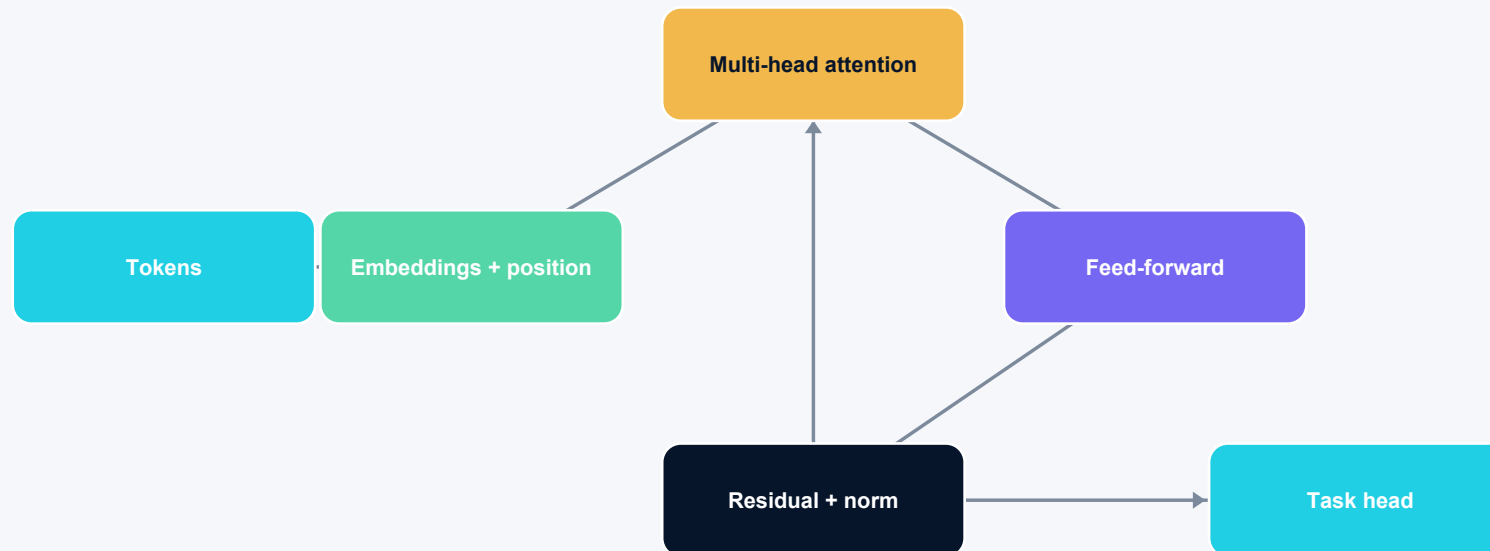


Sequence order, masks, length policy, and state boundaries are part of the model contract.

TEACHING PROMPT

For each arrow: name what moves, its shape or schema, who owns it, how it can fail, and which measurement proves the handoff works.

TRANSFORMER ATTENTION STACK

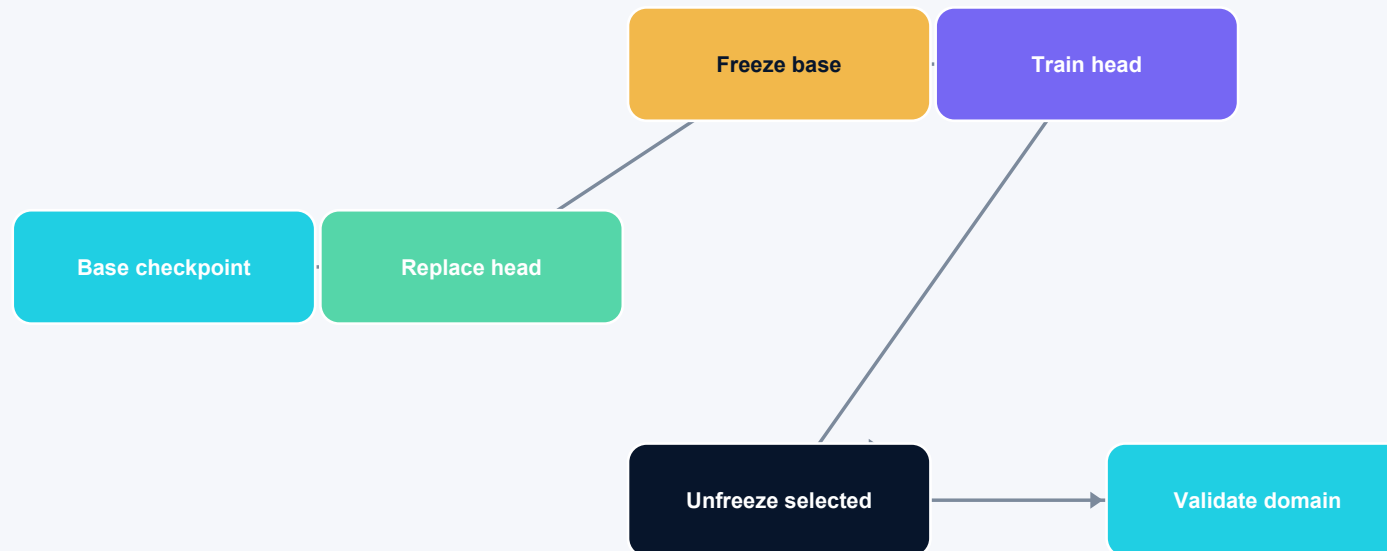


Attention mixes context; residual blocks and positional information preserve a trainable token representation.

TEACHING PROMPT

For each arrow: name what moves, its shape or schema, who owns it, how it can fail, and which measurement proves the handoff works.

TRANSFER LEARNING STRATEGY

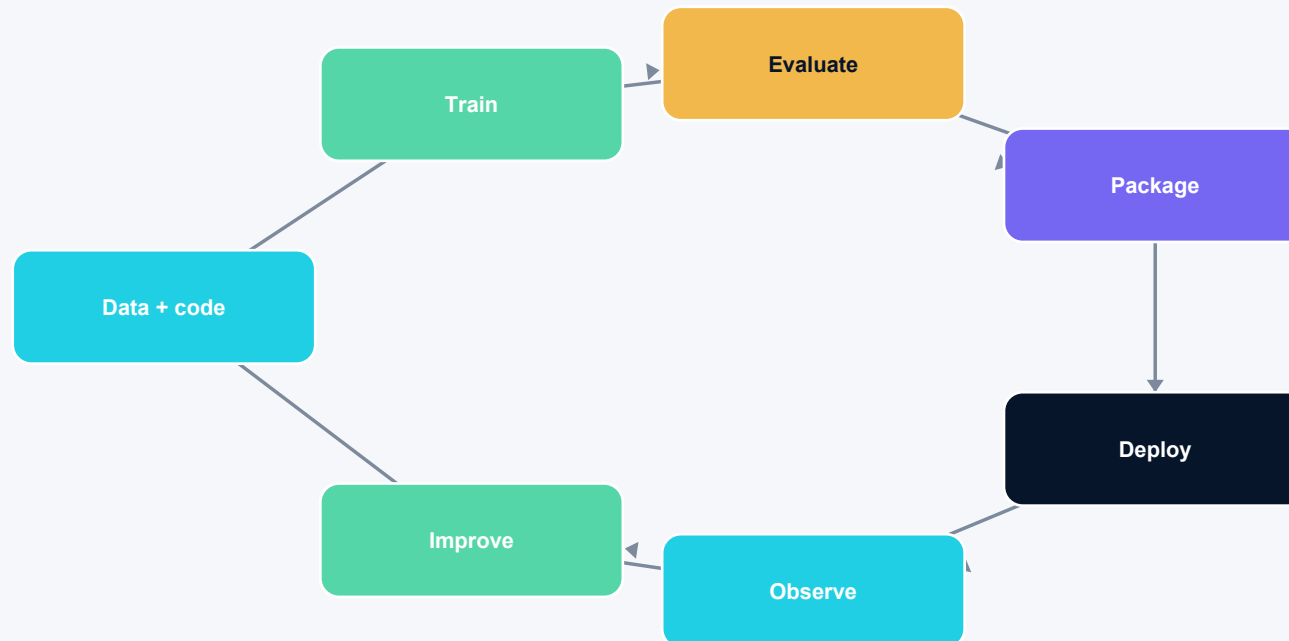


Adapt gradually and use domain-aware validation to decide whether representation reuse is helping.

TEACHING PROMPT

For each arrow: name what moves, its shape or schema, who owns it, how it can fail, and which measurement proves the handoff works.

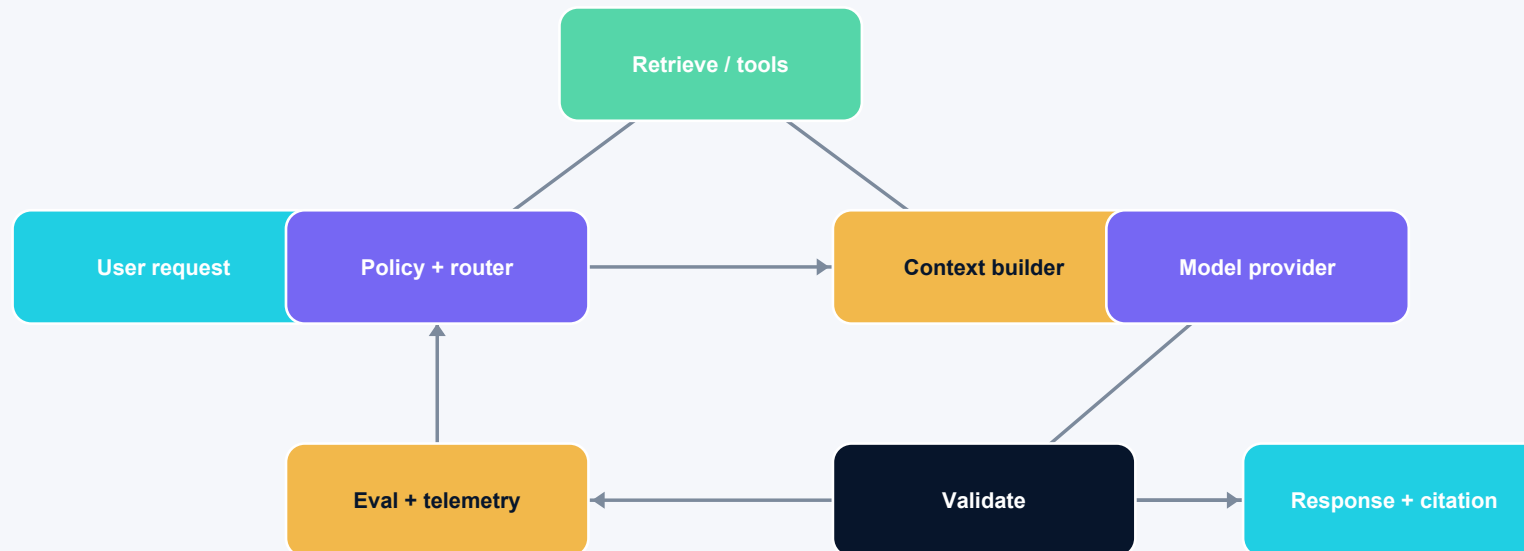
MLOPS LIFECYCLE



Versioned inputs, automated gates, deployment evidence, and feedback connect the model lifecycle.

TEACHING PROMPT

For each arrow: name what moves, its shape or schema, who owns it, how it can fail, and which measurement proves the handoff works.



Reliable generative AI separates provider calls, context, tools, validation, evaluation, safety, and observability.

TEACHING PROMPT

For each arrow: name what moves, its shape or schema, who owns it, how it can fail, and which measurement proves the handoff works.

VISUAL REVIEW SPRINT

Choose one project from the course. Combine the relevant diagrams into a single page, remove every component you cannot justify, then annotate the remaining interfaces with versions, schemas, metrics, and failure behavior.

